Proyecto #1

Arreglos Paginados

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Algoritmos y Estructuras de Datos II (CE 1103)
I Semestre 2026

OBJETIVOS

GENERAL

- Desarrollar una aplicación sencilla utilizando el paradigma de Programación Orientado a Objetos en C++ aplicando el concepto de paginación de administración de memoria

ESPECÍFICOS

- Familiarizarse con C++, Programación Orientada a Objetos y conceptos de administración de memoria

DESCRIPCIÓN GENERAL

En este proyecto, se le solicita al/la estudiante que implemente una clase PagedArray que permita manejar arreglos de gran tamaño sin mantener todo su contenido en la memoria principal. El proyecto incluye varios componentes relevantes:

1. Generador de archivos de prueba: archivos con números aleatorios en formato binario que serán ordenados por **algoritmos de ordenamiento estándar**.
2. Programa que realiza el ordenamiento utilizando la clase PagedArray que encapsula completamente el funcionamiento de un arreglo paginado, en el que solo ciertas partes del arreglo están cargadas en la memoria principal, mientras que el resto se encuentra en disco.

REQUERIMIENTOS DEL GENERADOR DE ARCHIVOS

Desarrollar un programa en C++ para generar archivos **binarios** grandes que contienen una cantidad predeterminada de números enteros **aleatorios**. Este programa se llamará *generator* y se ejecuta en la terminal de la siguiente forma:

```
$> generator -size <SIZE> -output <OUTPUT FILE PATH>
```

El argumento <SIZE> puede ser uno de los siguientes valores:

1. SMALL: genera un archivo de 512 MB
2. MEDIUM: genera un archivo de 1 GB
3. LARGE: genera un archivo 2 GB

<OUTPUT FILE PATH> corresponde a la ruta del sistema de archivos donde se guarda el archivo generador.

El contenido del archivo generado por la herramienta **no es legible, es totalmente binario** (ni siquiera se puede ver los caracteres 0 o 1, no tiene codificación). Cada aleatorio número generado, se guarda en el archivo en formato binario, es decir, cada 4 bytes escritos corresponden a un entero. No se debe separar por comas u otro separador. Es una simple serialización de enteros depositados en un archivo.

REQUERIMIENTOS DEL ORDENADOR DE ARCHIVOS

Desarrollar un programa en C++ para ordenar archivos **binarios** grandes que contienen una cantidad predeterminada de números enteros **aleatorios**. Este programa se llamará *sorter* y se ejecuta en la terminal de la siguiente forma:

```
$> sorter –input <INPUT FILE PATH> -output <OUTPUT FILE PATH> -alg
<ALGORITMO> -pageSize <PAGE-SIZE> -pageCount <PAGE-COUNT>
```

- El argumento <INPUT FILE PATH> es la ruta del sistema de archivos donde está el archivo por ordenar que contiene los números enteros aleatorios en binario.
- El argumento <OUTPUT FILE PATH> es la ruta del sistema de archivos donde se guarda el archivo **ordenado** resultante. Genera dos archivos, uno en binario y uno legible en ASCII o Unicode.
- El argumento <ALGORITMO> corresponde al nombre de 5 posibles algoritmos de ordenamiento soportados por el programa. Dado que algunos algoritmos de ordenamiento son inviables para archivos muy grandes, puede limitar y validar la entrada.
- El argumento <PAGE-SIZE> corresponde a la cantidad máxima de enteros de cada página.
- El argumento <PAGE-COUNT> corresponde a la cantidad de páginas que se tendrán en memoria

Internamente, *sorter* utiliza una clase llamada *PagedArray* que deberá ser implementada por el/la estudiante. La clase *PagedArray* funciona de la siguiente manera:

1. Utiliza máximo <Page-Count> arreglos de enteros (los usuales provistos por C++) . Estos corresponden a las únicas n páginas (o espacios para cargar páginas) de enteros que se pueden tener en memoria.

2. Sobrecarga el operador [] para proveer una abstracción familiar a un arreglo normal. Por ejemplo:

```
PagedArray* arr = new PagedArray();
arr[1000] = 1;
cout << arr[5000];
```

3. Cuando se procesa la operación arr[*indice*] se sigue la siguiente lógica:
    a. Calcular la página correspondiente donde el índice debería estar.
    b. Si la página está cargada en alguno de los n espacios de páginas, se retorna el valor de arr[indice] directamente desde la memoria.
    c. Si la página no está cargada, se carga desde el disco, desde el archivo que se está ordenando. Para esto, se puede usar la operación fseek() (o similar), junto con algunos cálculos relevantes. Si todos los espacios para páginas están ocupados, se emplea un algoritmo de reemplazo (no se acepta random) y se descarga al disco en su posición correspondiente.
    d. *sorter* no modifica el archivo especificado en el argumento <INPUT FILE PATH>, sino que lo copia a <OUTPUT FILE PATH> y trabaja sobre <OUTPUT FILE PATH> version binaria. Al final, genera un version legible basado en la version binaria

El código de los algoritmos de ordenamiento no debe conocer de la existencia de la lógica de paginación, es decir, recibirá un PagedArray como argumento, y funcionará de la forma usual como si usara un array normal. PagedArray debe asegurar encapsula correctamente toda la lógica de paginación, por ejemplo, considere el código **usual** de quicksort:

```
#include <iostream>
#include <vector>
#include <algorithm> // Para std::swap

// Función para particionar el arreglo
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high]; // Seleccionar el último elemento como pivote
    int i = (low - 1); // Índice del elemento más pequeño

    for (int j = low; j <= high - 1; j++) {
        // Si el elemento actual es menor que el pivote
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
```

```
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

// Función principal de Quicksort
void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        // pi es el índice de partición, arr[pi] está ahora en el lugar correcto
        int pi = partition(arr, low, high);

        // Ordenar los elementos antes y después de la partición
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

El único cambio permitido al código anterior sería lo siguiente:

```
...

// Función para particionar el arreglo
int partition(PagedArray& arr, int low, int high) {
   ...
}

// Función principal de Quicksort
void quickSort(PagedArray& arr, int low, int high) {
    ...
}
```

Al finalizar la ejecución de sorter, se imprime un resumen de:

1. Tiempo durado
2. Algoritmo utilizado
3. Cantidad de page faults vs page hits

Un *page fault* corresponde a cada vez que se buscó un elemento y no estaba en las páginas cargadas y **se tuvo que cargar dicha página desde el disco**. Un *page hit*, a cada vez que se buscó un elemento y estaba en las páginas cargadas y **no se tuvo que cargar dicha página desde el disco**

El archivo <OUTPUT FILE PATH> no-binario deberá estar correctamente ordenado ascendentemente y **debe ser legible para poder verificar la correctitud. Deben usar comas para separar los enteros.**

DOCUMENTACIÓN

Se entregan dos componentes de documentación:

1. En el repo de GitHub, el archivo README debe incluir instrucciones claras de como ejecutar el generador y el ordenador.
2. **Se entregará un *paper* en formato PDF** utilizando la plantilla de IEEE donde se incluya:
    a. Contexto del problema
    b. Detalles de implementación indicando cualquier limitante de implementación.
    c. Detalles de experimentación: cuales algoritmos de ordenamiento y reemplazo utilizará
    d. Resultados de experimentación incluyendo gráficas de page faults/page hits durante el ordenamiento con distintos tamaños de página y para cada algoritmo, tiempos de ordenamiento y cualquier otro dato relevante que quieran explorar.
    e. Conclusiones sobre el proyecto, oportunidades de mejora
    f. Link del Repositorio de Github
    g. Anexo: prompts utilizados para agentes de AI

Se evaluará redacción, ortografía y profundidad del paper.

ASPECTOS OPERATIVOS

- La fecha de entrega será según lo especificado en el TEC Digital. No se aceptan entregas tardías. No hay excepciones.
- El trabajo se puede realizar únicamente de forma individual
- El uso de Git y Github es obligatorio
    - Incluir todos los archivos fuente, headers y archivos de construcción
    - Incluir instrucciones para compilar, ejecutar y probar el código.
    - En el repo NO se incluyen archivos binarios, ejecutables o compilados.
- La revisión de la documentación será realizada por parte del profesor, no durante la defensa del proyecto.
- Cada persona tendrá 10 minutos para exponer su trabajo al profesor y defenderlo, es responsabilidad de los estudiantes mostrar todo el trabajo realizado, por lo que se recomienda tener todo listo antes de entrar a la defensa.
- **El uso de la clase vector o cualquier otro "arreglo" no estándar está probihido. Equivale a nota cero. Es mejor validar con el profesor antes de usar dicha clase**

EVALUACIÓN

- El proyecto tiene un valor de 15% de la nota del curso
- Si los componentes del proyecto no están integrados, es decir, “todo sirve por aparte, pero nada en conjunto”, la nota total del proyecto será cero.
- Si durante la defensa, cualquier miembro del equipo es incapaz de explicar qué hace **cualquier** parte del código, la nota de proyecto será cero.

- Si se detecta el uso de AI pero no se entregan los *prompts* utilizados para los agentes de AI, obtienen nota de cero.
- Cada mala práctica de programación detectada se penaliza con 1 punto.
- La rúbrica es la siguientes:

Componente	Peso
Generador de archivos desordenados	**10%**
- Validar argumentos de la línea de comandos (5 pts) - Soporte de todas las opciones (5 pts) - Eficiencia de la generación (20 pts) - Genera archivos en binario correctamente (80 pts)	
Ordenador de archivos	**80%**
- Implementación de todos los algoritmos sin modificar su código a excepción del uso de Paged Array en la firma del procedimiento (25 pts) - Uso de memoria que refleje la paginación correcta (25 pts) - Algoritmos de reemplazo (20 pts) - Output generado (5 pts) - Funcionamiento correcto del ordenamiento (25 pts)	
Documentación	**10%**
• Readme en el repo (5pts) • Paper con formato correcto y contenido esperado según lo solicitado (95)	

- A los 5 proyectos más eficientes en memoria y tiempo de ejecución, se les dará 10 puntos más, siempre y cuando el resto de su implementación esté completa y correcta.